Università degli Studi di Messina
Dipartimento MIFT - Corso di Laurea Triennale in Informatica (L-31)

Laboratorio di Reti e Sistemi Distribuiti

HANDS-ON 17:
Algoritmo di Bellman-Ford per il Distance-Vector Routing

Massimo Mantineo
Matricola: 541924

Messina, A.A. 2024/2025

Indice

1 Problem Statement **2**

2 Stato dell'Arte **2**

3 Metodologia ed Implementazione **2**
3.1 Istruzioni di Esecuzione 3

4 Risultati **3**

5 Conclusioni e Sviluppi Futuri **4**

1 Problem Statement

Nell'ambito dell'astrazione logica di nodi di rete come processi UNIX, si richiede l'implementazione del celebre algoritmo matematico di **Bellman-Ford** per il monitoraggio dei grafi pesati. Si tratta dell'archetipo fondante per la classe di protocolli *Distance-Vector* usati dai router fisici per eleggere la minima distanza hop-by-hop verso una sottorete (Si veda, ad esempio, lo standard 'RIP' o 'EIGRP'). Ancora una volta l'elaborazione non sarà accentrata su un database monolitico for-loop classico, ma si propagherà in maniera genuinamente **Distribuita ed Asincrona** sfruttando `fork` ed appoggiandosi alle Syscall `mkfifo` per le code di messaggistica.

2 Stato dell'Arte

L'algoritmo di Bellman-Ford assicura, in reti prive di cicli a costo negativo, la convergenza totale dei pesi. Esso si esprime tramite l'assioma di calcolo iterativo:

$$D(x, y) = \min_{v} \{c(x, v) + D(v, y)\} \tag{1}$$

Dove:

- $D(x, y)$: Il valore stimato attualizzato della distanza dal nodo x al target y.
- $c(x, v)$: Il peso prefissato nel cavo (Edge) che separa il nodo adiacente x dal nodo limitrofo v.

Ogni nodo *riceve* in asincrono messaggi in buffer dalla propria FIFO, decodifica il Vettore $D(v, y)$ ricevuto, somma all'overhead biologico $c(x, v)$ che esso prova per raggiungere v, e decreta se il cammino calcolato è più economico di quello che custodiva in memoria, mutando ed allertando i vicini di rimando in caso di Update positivo.

3 Metodologia ed Implementazione

Siccome il grafo visivo manca all'appello, optiamo per disporre una trincea a **Cost Matrix Asimmetrica** per 4 Nodi:

```
// Single-Source cost_matrix
// Node 0 is the Target.
int cost_matrix[NUM_NODES][NUM_NODES] = {
    {0, 2, 5, INF}, // Edge out dal nodo 0
    {2, 0, 1, 4},   // Edge out dal nodo 1
    {5, 1, 0, 1},   // Edge out dal nodo 2
    {INF, 4, 1, 0}  // Edge out dal nodo 3
};
```

Abbiamo programmato volontariamente un trabocchetto: benché esista l'arco diretto Nodo 2 $\rightarrow$ Target 0 al costo di asticella '5', la somma dei pesi passando attivamente per Nodo 1 costa solamente '3' $(1 + 2)$. Costringiamo così in maniera fittizia l'algoritmo a

ricalcolare le metriche ignorando l'arco apparentemente più palese (Il nodo 3 dista '1' dal 2, per un totale di '4', ed in quanto esso dista '4' dal Nodo 1 che aggiunge il fatidico '2', la via più ovvia peserebbe '6'. Viene scelto '4').

3.1 Istruzioni di Esecuzione

L'applicativo viene gestito interamente tramite Makefile:

Build Run

```
$ make
$ ./programma
```

4 Risultati

Si è provveduto allo sfoggio del binario, compilato per iterazioni continue ed abortito dopo 3_{sec} dall'incessazione delle scoperte IPC (Timeout Convergenza).

Mutua Esclusione Distribuita - Output

```
massimo@PORTATILE-Linux:~/Scrivania/LabReSid24-25/hands-on/ho17$
   ./bellman_ford
===== Bellman-Ford Algoritmo di Distance Vector Distribuito
   =====
Obiettivo: Cammini ottimali da ogni Nodo verso il NODO
   TARGET (0)

[NODO 0 - TARGET] Distanza zero locale. Segnalo Inizio alle
   adiacenze.
[NODO 1] D-V UPDATE! Vett. 999 > 2 (Nuovo hop: Nodo 0).
   Espando l'onda d'urto...
[NODO 2] D-V UPDATE! Vett. 999 > 5 (Nuovo hop: Nodo 0).
   Espando l'onda d'urto...
[NODO 3] D-V UPDATE! Vett. 999 > 6 (Nuovo hop: Nodo 1).
   Espando l'onda d'urto...
[NODO 2] D-V UPDATE! Vett. 5 > 3 (Nuovo hop: Nodo 1).
   Espando l'onda d'urto...
[NODO 3] D-V UPDATE! Vett. 6 > 4 (Nuovo hop: Nodo 2).
   Espando l'onda d'urto...

--- [NODO 2 ESTINTO/CONVERSO] Shortest Path Totale => 3 (via
   Nodo 1) ---
--- [NODO 0 ESTINTO/CONVERSO] Shortest Path Totale => 0 (via
   Nodo 0) ---
--- [NODO 3 ESTINTO/CONVERSO] Shortest Path Totale => 4 (via
   Nodo 2) ---
--- [NODO 1 ESTINTO/CONVERSO] Shortest Path Totale => 2 (via
   Nodo 0) ---

==== Convergenza Assoluta Raggiunta. Discesa IPC ====
```

5 Conclusioni e Sviluppi Futuri

La traccia IPC e i log confermano clamorosamente le nostre predisposizioni da scrivania. All'insorgere del tempo e con l'avanzamento dei cicli, il **NODO 2 e il NODO 3 scartano le rotte precedenti (5 e 6 rispettivamente)** per abbracciare l'Update di vettori asincrono che segnalava la presenza del percorso a costo calcolato '3' e '4', dimostrando ad amplifero la correttezza del demone C. Le sysfile temporanee di buffer (Pipe multiple) vengono annibillate dolcemente dallo sterrato in chiusura dell'ambiente Unix locale.